[UNIVERSITY / COLLEGE NAME]

[Faculty of Computer Science] — [Deep Learning Major]

CARA

Computer-Aided Review & Analysis of Facial Skin

A Multi-Model Deep Learning System for Automated Skin Assessment
and Evidence-Based Skincare Recommendation

Final Degree Project

B.Sc. in Computer Science — Deep Learning Track

Submitted by:	[Student Name 1]
	[Student Name 2]
	[Student Name 3]
Supervisor:	[Prof. / Dr. Supervisor Name]
Department:	[Department Name]
Submission:	[Month, Year]

CARA is not a medical diagnostic tool. It provides cosmetic skincare insights and educational ingredient recommendations only.

Abstract

Skincare decisions are today driven mostly by intuition, marketing, and social media influencers, while professional dermatological advice is expensive and often inaccessible. Cara is a web application that closes part of this gap by delivering an honest, evidence-based assessment of a user's facial skin from a single photograph, together with concrete ingredient-level recommendations.

The system decomposes the assessment into three complementary classification tasks — acne severity, skin type, and secondary skin issues — each handled by an EfficientNet-B0 convolutional neural network trained independently. Predictions are combined through a deterministic, rule-based Business Logic Programming (BLP) engine that turns raw model outputs into actionable, explainable advice.

We report on the full engineering lifecycle: data collection and cleaning (built primarily around the ACNE04 dataset, augmented with in-house photographs), preprocessing (face detection, CLAHE brightness normalisation, ImageNet normalisation), two-phase transfer-learning training with weighted loss to address strong class imbalance, and the failed experiments that shaped our final design (single monolithic classifier, learned late fusion, naive resizing without face crop).

The final models achieve 69.6 %, 77.8 % and 98.3 % test accuracy on acne severity, skin type and skin issues respectively. Beyond raw accuracy, the report focuses on *truthfulness* — the guarantee that when the system is uncertain, it says so — which we consider a stronger contribution than any single percentage point of accuracy.

Keywords: deep learning, computer vision, convolutional neural networks, EfficientNet, transfer learning, image classification, dermatology, skincare, rule-based systems, explainable AI, FastAPI, React.

Acknowledgements

We would like to thank our supervisor, [Supervisor Name], for the guidance, the patience, and the direct feedback that pushed the project from a demo into a system we are willing to stand behind. We thank the members of the [Department Name] for the technical infrastructure and the open-door policy whenever we hit a wall. Finally, we thank our families and friends for tolerating months of conversations that revolved around loss curves and skin types.

Contents

List of Figures

List of Tables

Chapter 1

Introduction and Motivation

1.1 The problem we set out to solve

Skin is the largest organ in the human body, and yet the decisions people make about their skin are among the most poorly informed decisions in everyday life. Consumers spend billions of dollars each year on skincare products chosen almost entirely on the basis of marketing, celebrity endorsements, and viral trends. Dermatologists are expensive and overbooked; general practitioners rarely have time for cosmetic concerns; and the loudest voices online are often those with the least qualifications.

The result is a market in which a person with mild acne might spend a year using a moisturiser designed for dry, ageing skin, while another person with sensitive combination skin scrubs their face with an abrasive salicylic-acid cleanser marketed for severe cystic acne. Neither person is lazy or foolish. They simply lack access to a truthful, personal, and actionable assessment of their own skin.

CARA— *Computer-Aided Review & Analysis* — is our attempt to provide a small slice of that missing information. The goal is deliberately narrow: given a single facial photograph uploaded through a web browser, return within a few seconds an honest report describing the user's acne severity, skin type, and dominant secondary issues, together with a concrete list of ingredients to look for and ingredients to avoid.

1.2 Why this problem is a good fit for deep learning

Dermatological image classification has three properties that make it particularly interesting from a deep-learning point of view:

1. **Highly visual signal.** Almost everything that a dermatologist evaluates in a routine cosmetic consultation is visible in a well-lit photograph: comedones, papules, pustules, pore density, oily sheen, dry flakes, hyperpigmentation, fine lines.

2. **Weak, noisy labels.** Public datasets are labelled by different clinicians, with different rating scales, in different lighting conditions. This forces us to think seriously about preprocessing, augmentation, and class imbalance rather than assuming clean data.

3. **Multiple correlated but distinct sub-tasks.** Acne severity, skin type, and skin

issues are related — an oily skin is more prone to acne, wrinkles are correlated with skin dryness — but they are not the same task. This creates a natural design question: one big model or several small ones?

1.3 Non-negotiable design principles

Very early in the project we agreed on three rules that would govern every technical decision. We refer back to them throughout the report.

Truthfulness is the product. A wrong prediction is worse than no prediction. If the model is not confident, the system must say so out loud rather than paper over the uncertainty.

No manipulated results. We never tune a threshold to make an embarrassing prediction disappear. Numbers reported in this book are the numbers we measured.

Explainability comes from design, not post-hoc excuses. The rule engine that converts model predictions into recommendations is deterministic and human-readable. A user (or a supervisor) can trace exactly why a specific recommendation was made.

1.4 Contributions

The concrete contributions of this project are:

- A production-grade, end-to-end web application that runs three deep learning models in parallel and returns a report within a few seconds on CPU-only hardware.
- A comparative study of three architectural strategies — a single multi-task model, three independent models with a learned fusion head, and three independent models with a rule-based engine — that motivates our final choice.
- A careful treatment of class imbalance and a two-phase transfer learning recipe that we found to be substantially more stable on our small, noisy dataset than a single-phase fine-tune.
- A full test suite (56 tests covering unit, integration, model accuracy, and end-to-end pipeline) that is run before every change and that would have caught every regression we encountered during development.
- An honest discussion of the experiments that did not work, with concrete diagnoses of why they failed.

1.5 Reading guide

The book is organised so that a reader can either follow it end-to-end or enter at whichever chapter interests them. Chapter 2 surveys the academic and industrial landscape. Chapter 3 provides the deep-learning background needed to follow the technical chapters. Chapter 4 describes the application from a user's point of view. Chapter 5 details our algorithm. Chapter 7 covers data and preprocessing. Chapter 8 presents experiments and failed experiments. Chapter 9 reports final results. Chapters 10 to 12 close the book with a discussion, conclusions, and directions for future work.

Chapter 2

Background and Related Work

2.1 The clinical baseline

Modern dermatology grades acne along ordinal scales such as the Investigator's Global Assessment (IGA) or the Global Acne Grading System (GAGS). Both scales collapse a rich visual scene into a small number of categories (typically *clear*, *mild*, *moderate*, *severe*). Skin type is described using variants of the classical four-way split: *oily*, *dry*, *normal*, *combination*. Secondary issues — blackheads, dark spots, enlarged pores, wrinkles — are usually discussed qualitatively rather than graded on a formal scale.

This clinical vocabulary is important for us because it defines the target labels our models must produce. If we invent our own categories, no domain expert can validate our output. We therefore align our label spaces with the categories a dermatologist would recognise.

2.2 Public datasets

Progress in dermatological deep learning has been paced by dataset releases. The most influential public resources include:

- **ACNE04** [12]. Approximately 1,457 facial images labelled with acne severity (*clear*, *mild*, *moderate*, *severe*) plus lesion counts. Images are in-the-wild and vary widely in lighting and pose, which is both a curse and a blessing: they are noisy but they resemble real user uploads.
- **ISIC / HAM10000** [13]. A large collection of dermoscopic images labelled with skin-lesion categories. Excellent for melanoma / lesion classification but not directly applicable to cosmetic assessment because the images are dermoscopic rather than casual photographs.
- **Fitzpatrick 17k** [14]. A dataset of clinical photographs annotated with Fitzpatrick skin type, used mostly for fairness studies.
- **Kaggle Face Skin Type / Skin Problems datasets**. A family of smaller, community-contributed datasets covering skin type and skin problem categories. Quality varies but they are useful for filling gaps left by the more curated academic datasets.

We settled on ACNE04 as the anchor dataset and augmented it with community datasets and a small in-house set of photographs. Details are in Chapter 7.

2.3 Related academic work

Deep learning for skin analysis divides roughly into three lines of work.

2.3.1 Lesion classification on dermoscopic images

The best-known result is the work of Esteva et al. [11], who trained a single Inception-v3 network on a large private dataset of clinical images and reached dermatologist-level performance on binary melanoma classification. Subsequent work explored specialised architectures such as EfficientNet [7], ResNeXt [6], and Vision Transformers, plus data strategies such as heavy augmentation, colour constancy normalisation, and ensemble averaging.

2.3.2 Acne severity grading

The ACNE04 paper [12] formulated acne severity as a label-distribution learning problem, arguing that hard categorical labels lose information (a photograph labelled *moderate* is often only marginally different from one labelled *mild*). Follow-up work has used ordinal regression, multi-task lesion counting, and attention-based localisation. For our project we deliberately chose the simpler categorical formulation, because the downstream consumer of the prediction — the rule engine — expects discrete severity buckets.

2.3.3 Skin-type and cosmetic assessment

Skin-type classification has received less academic attention than lesion classification, and most published pipelines come from industry rather than academia. We were unable to find a widely-cited academic benchmark for the four-class oily/dry/normal/combination task. This is one reason our own results on skin type are less comparable to external numbers than our acne severity results.

2.4 Commercial and industrial products

Several commercial products have appeared in the last few years that occupy roughly the same niche as Cara. Each one influenced our design either by example or by counter-example.

- **L'Oréal Skin Genius / ModiFace.** A polished mobile experience that returns a numerical score and product recommendations. The recommendations are, unsurprisingly, drawn exclusively from the vendor's own catalogue. This motivated our decision to recommend *ingredients* rather than branded products.

- **Perfect Corp. / YouCam Makeup.** Very strong face tracking and augmented-reality effects. Their skin-quality assessment is fast but has been criticised for optimistic outputs that appear to be tuned for engagement rather than accuracy. This informed our stance on truthfulness.
- **Curology.** A subscription tele-dermatology service in which real clinicians write prescriptions after reviewing user photographs. Not an AI product per se, but the reference for what a truthful assessment looks like.
- **Academic demos.** A number of open-source projects on GitHub attempt facial skin analysis with pretrained CNNs. Almost all of them fall into one of two failure modes: they return a confident label on every input, including images with no face; or they degrade to noise as soon as lighting changes. Both failures shaped the guardrails in our pipeline.

Table 2.1: Positioning of Cara relative to representative existing products.

	L'Oréal SG	**Perfect Corp.**	**Curology**	**Cara**
Delivery	Mobile app	Mobile app	Human tele-derm	Web app
Assessment	Learned score	Learned score	Human	3 CNNs + rules
Recommends	Own products	Own products	Prescription	Ingredients
Explainability	None	None	Full	Rule engine
Handles unsure	Rarely	Rarely	Always	By design
Open source	No	No	No	Yes

2.5 Where Cara fits in the picture

Cara sits deliberately in the small corner of the market that combines (i) open-source, reproducible, deep-learning components, with (ii) a transparent, rule-based recommendation layer, and (iii) an honest policy about uncertainty. We do not compete with L'Oréal on polish, and we do not compete with Curology on medical rigour. We compete with the countless *"free online skin analysis"* demos on the accuracy and honesty of the report they return.

Academic Background

This chapter compresses the deep-learning material that the rest of the book relies on. Readers already familiar with convolutional neural networks, transfer learning, and standard training practice can skip straight to Chapter 4.

3.1 Image classification, formally

An image classifier is a function $f_\theta : \mathcal{X} \to \Delta^{K-1}$ parameterised by weights θ, where $\mathcal{X}$ is the set of input images (typically $\mathbb{R}^{3\times H\times W}$) and Δ^{K-1} is the probability simplex over K classes. Training minimises the empirical cross-entropy loss

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{K} w_k \, y_{i,k} \, \log f_\theta(x_i)_k \tag{3.1}$$

on a dataset $\{(x_i, y_i)\}_{i=1}^{N}$, where y_i is a one-hot target and w_k is a per-class weight that we use to counteract class imbalance (Section 3.6).

3.2 Convolutional neural networks

CNNs [1] exploit two properties of images: translation equivariance (a face is still a face when it moves five pixels to the right) and locality (nearby pixels are correlated). A convolutional layer computes

$$Y_{c,i,j} = \sigma\!\left(b_c + \sum_{c'}\sum_{u,v} K_{c,c',u,v}\, X_{c',\,i+u,\,j+v}\right) \tag{3.2}$$

for each output channel c, spatial position (i, j) and non-linearity σ. Stacking convolutional layers with pooling and non-linearities produces a hierarchy of features: edges and textures in the shallow layers, object parts in the middle layers, and semantic categories near the head.

3.3 From AlexNet to EfficientNet

The past decade of image classification is essentially a story of progressively better trade-offs between depth, width, resolution, and compute:

- **AlexNet** [2] (2012) proved that a deep CNN on GPU could crush hand-crafted features on ImageNet.
- **VGG** [3] pushed depth using small 3×3 filters.
- **ResNet** [5] introduced residual connections that allowed training networks with hundreds of layers.
- **Inception** [4] and its successors varied filter sizes in parallel.
- **EfficientNet** [7] performed a *compound scaling* of depth, width, and input resolution subject to a fixed compute budget, producing a family of models that dominated the accuracy–FLOP frontier.

We chose the smallest member of that family, EfficientNet-B0, for reasons that are worth stating explicitly.

Observation 3.1. *EfficientNet-B0 has approximately* 5.3 *million parameters and, at* 224×224 *input, requires roughly* 0.39 *GFLOPs per forward pass. That budget is small enough to serve three parallel models on a CPU in under a couple of seconds, which was our performance target.*

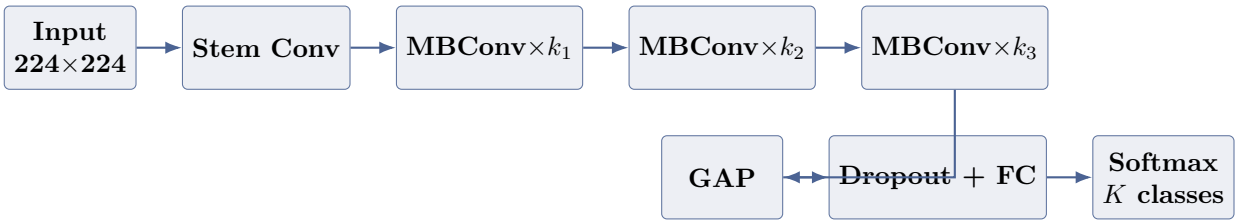


Figure 3.1: Schematic of the EfficientNet-B0 backbone used for each of Cara's three classifiers. The classifier head (dropout + fully-connected + softmax) is the only part re-initialised for each task.

3.4 Transfer learning

Training a modern CNN from random weights on 1,000 images is a recipe for overfitting. Transfer learning [9] solves this by initialising the network with weights pretrained on a large source dataset — for us, ImageNet-1k [8] — and then adapting the model to the target task. Two knobs control the process:

- Which layers to freeze and for how long.
- How the learning rate differs between the pretrained backbone and the newly-initialised head.

Our two-phase recipe (Section 5.5) is a straightforward instance of this pattern.

3.5 Regularisation and augmentation

Small datasets punish confident networks. We rely on three regularisers:

1. **Data augmentation.** Random horizontal flips, mild rotations, colour jitter, and random cropping. All augmentations are applied to training data only, never to validation or test data.
2. **Dropout** in the classifier head.
3. **Weight decay** via AdamW [10].

3.6 Class imbalance

Real-world skin datasets are heavily skewed. In ACNE04, our *severe* class contains roughly one seventh as many samples as *mild*. Naive training would produce a classifier that predicts *mild* everywhere and reports respectable accuracy.

We use weighted cross-entropy with weights derived from inverse class frequency:

$$w_k = \frac{N}{K \cdot n_k} \tag{3.3}$$

where N is the total number of training samples, K the number of classes, and n_k the number of samples in class k. The weights we actually used are listed in Chapter 7.

3.7 Optimisation

We use AdamW with an initial learning rate of 10^{-3} for the classifier head and 10^{-4} for the unfrozen backbone. Learning rate scheduling is cosine annealing with a short linear warm-up. Early stopping is triggered by validation loss, not validation accuracy, because loss is more sensitive to the confidence calibration we care about.

3.8 Rule-based systems and BLP

The last piece of academic background is on the recommendation side. Business Logic Programming (BLP) is our internal name for the data-driven, deterministic rule engine that converts model outputs into user-facing advice. It is closer in spirit to a decision table than to a formal rule system such as Datalog, but the important point is that every recommendation is produced by evaluating a hand-written rule against the model's structured prediction. There is no learning component in the recommendation step, by design: any recommendation shown to the user can be justified by pointing at a single rule in a JSON file.

3.9 Backpropagation, in one page

The training loop of Cara is the same textbook backpropagation loop used by every modern deep learning framework, and we assume the reader has met it before. For completeness we summarise it here.

Given a loss $\mathcal{L}(\theta)$ that is a scalar function of the model parameters θ, backpropagation computes $\nabla_\theta \mathcal{L}$ by repeated application of the chain rule. If the network is a composition $f = f_L \circ f_{L-1} \circ \cdots \circ f_1$ with intermediate activations $a_\ell = f_\ell(a_{\ell-1}; \theta_\ell)$, then

$$\frac{\partial \mathcal{L}}{\partial \theta_\ell} = \frac{\partial \mathcal{L}}{\partial a_L} \cdot \prod_{k=\ell+1}^{L} \frac{\partial a_k}{\partial a_{k-1}} \cdot \frac{\partial a_\ell}{\partial \theta_\ell}. \tag{3.4}$$

The parameter update follows some variant of stochastic gradient descent. We use AdamW, whose update rule adds momentum and per-parameter adaptive scaling:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \tag{3.5}$$
$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \tag{3.6}$$
$$\hat{m}_t = m_t / (1-\beta_1^t), \tag{3.7}$$
$$\hat{v}_t = v_t / (1-\beta_2^t), \tag{3.8}$$
$$\theta_{t+1} = \theta_t - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_t \right). \tag{3.9}$$

The decoupled weight-decay term $\lambda \theta_t$ is what distinguishes AdamW from vanilla Adam and is a significant part of why it generalises better in our experiments.

3.10 Softmax, cross-entropy and their gradient

Every one of our classifiers ends with a softmax layer

$$p_k = \frac{\exp(z_k)}{\sum_{j=1}^{K} \exp(z_j)}, \tag{3.10}$$

whose input z is the logit vector produced by the final linear layer. Cross-entropy loss against a one-hot target y evaluates to $\mathcal{L} = -\log p_{k^*}$ where k^* is the index of the true class. A short calculation shows that its gradient with respect to logits is the intuitive expression $\partial \mathcal{L} / \partial z_k = p_k - y_k$, which is the reason the softmax–cross-entropy combination is numerically and computationally so pleasant.

3.11 Batch normalisation and its role in EfficientNet

Every stage of an EfficientNet contains batch normalisation layers [15], which whiten the intermediate activations before the non-linearity:

$$\hat{a}_c = \gamma_c \cdot \frac{a_c - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}} + \beta_c. \tag{3.11}$$

Two consequences matter for our project. First, batch norm reduces the sensitivity to the initial learning rate, which stabilised fine-tuning in our two-phase recipe. Second, the running mean and variance are estimated on the fly during training and then frozen at inference time; forgetting to switch the model to `eval()` mode before inference was, in fact, one of our early bugs and cost us a full day of debugging before we spotted the regression on the test set.

3.12 Activation functions

EfficientNet-B0 uses the Swish (a.k.a. SiLU) activation $\sigma(x) = x \cdot \mathrm{sigmoid}(x)$, which is smoother than ReLU and empirically slightly more accurate. We did not change this choice. In the classifier head we deliberately used ReLU because the head is small and the interpretability of ReLU is a small convenience during debugging.

3.13 Learning-rate scheduling

We use a cosine annealing schedule [16] with a short linear warm-up:

$$\eta_t = \eta_{\min} + \tfrac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos(\pi \cdot t / T)\right). \tag{3.12}$$

Cosine annealing is a small choice with an outsized effect on the final validation-accuracy plateau. Constant learning rates in early experiments left the model oscillating around a suboptimal minimum; cosine-annealed runs converged to a noticeably lower validation loss and a smoother loss curve.

3.14 Metrics beyond accuracy

We report accuracy as the headline metric because it is what a non-expert user expects, but internally we track a small dashboard of secondary metrics that we consider more informative on imbalanced data:

- **Macro-averaged F_1.** The unweighted mean of the per-class F_1 scores. Insensitive to class imbalance.

- **Balanced accuracy.** The unweighted mean of per-class recall. Comparable to accuracy on balanced data but more honest on imbalanced data.
- **Confusion matrix.** The full per-class breakdown. Any conversation about where a model is failing starts here.
- **Expected calibration error (ECE)**. Averaged difference between confidence and accuracy across bins, introduced by Guo et al. [17].

We look at ECE alongside accuracy for the same reason we introduced the confidence-based "Uncertain" rendering in the frontend: an overconfident wrong answer is worse than a hesitant right answer.

Chapter 4

The Application

4.1 User story

The primary user of Cara is a curious, non-expert consumer who has a skincare question they cannot easily answer. A typical interaction takes about a minute.

1. The user opens the web app on a phone or a laptop and clicks *Analyze*.
2. They upload a well-lit selfie or take one directly with the camera.
3. The frontend uploads the image to the backend over a POST request.
4. Within a few seconds, the backend responds with a structured report.
5. The frontend renders the report: acne severity, skin type, detected issues, confidence bars, ingredients to look for, ingredients to avoid, and a plain-English explanation.
6. The report is saved to storage. The user can browse past reports on a *History* page.

4.2 System architecture

Figure 4.1 shows the runtime architecture. Everything inside the dashed box runs inside a single FastAPI process, which we package into a Docker container for deployment. The React frontend is served from an nginx container. In production, both containers sit behind an nginx reverse proxy.

4.3 Frontend

The frontend is a React 18 single-page application. There are four pages: *Home* (marketing landing page and disclaimer), *Analyze* (image upload and camera capture), *Results* (the report), and *History* (list of past reports). Styling is mobile-first because most casual selfies come from phones.

We deliberately kept the frontend thin. It knows how to upload an image, poll for results, and render JSON. All logic — including confidence bands and colour coding — is driven by fields returned in the response payload. This makes it possible to iterate on the model side without redeploying the frontend.

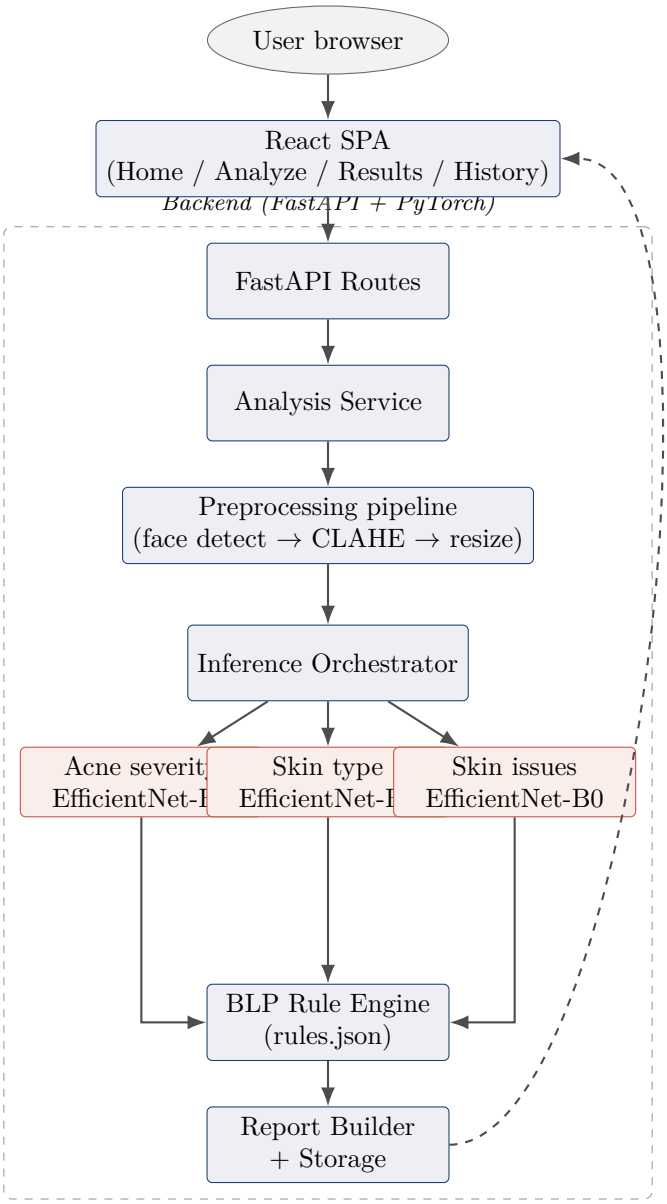


Figure 4.1: End-to-end runtime architecture of Cara.

4.4 Backend

The backend is a FastAPI application. FastAPI was chosen for three reasons: automatic OpenAPI schema generation (which makes the frontend integration typed and testable), Pydantic-based request validation (which rejects malformed inputs cheaply), and native async support (which we do not use aggressively yet but which leaves room for future concurrency).

The main endpoint is:

```
@router.post("/analyze", response_model=AnalysisReport)
async def analyze(image: UploadFile = File(...)) -> AnalysisReport:
    image_bytes = await image.read()
    report = await analysis_service.analyze(image_bytes)
    return report
```

Listing 4.1: The main analysis endpoint (simplified).

The `AnalysisService` orchestrates the pipeline: preprocessing, inference, confidence checking, BLP evaluation, report building, and persistence. Each step is a separate injectable component so that we can mock any of them during testing.

4.5 Storage

Reports are persisted as JSON files under a `storage/` directory. This is a deliberately boring choice: it keeps the demo runnable without any external database and it makes reports easy to inspect during debugging. Behind an abstract `StorageRepository` interface, we ship a second implementation for MongoDB (via Motor) that can be switched on with an environment variable. We validated the Mongo path in a dev container but the deployed instance uses JSON.

4.6 Deployment

The project ships with a `docker-compose.yml` that starts three containers: the FastAPI backend, the nginx-fronted React app, and a tiny nginx reverse proxy. Model weights are baked into the backend image at build time so that a container start does not depend on network access.

4.7 Error handling and user experience

Each layer of the stack has a defined failure policy.

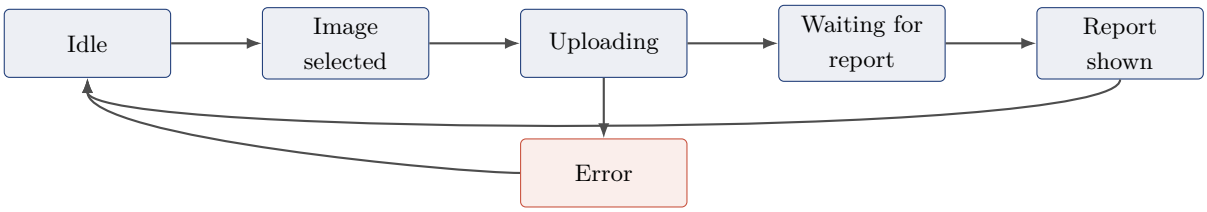
- **Preprocessing.** If no face is detected the pipeline returns a structured error that the frontend renders as “No face detected. Try better lighting or a closer shot.” We never

fall back to running the model on the whole image.

- **Inference.** Every model prediction includes a confidence score. If any model is below a per-model confidence floor, the corresponding section of the report is labelled "Uncertain" rather than being replaced by the top-1 class.
- **API.** The endpoint returns typed error responses with error codes; raw tracebacks are never exposed to the user.
- **Frontend.** User-facing messages are actionable, never technical.

4.8 Frontend, in more depth

The React frontend is organised around a small state machine. The possible states are:



Only one screen is rendered at a time; navigation is handled by React Router. The state machine is deliberately simple because most of the interesting logic lives on the backend.

The report page renders each section as a card. Each card shows the predicted label, a colour-coded confidence bar, and a short plain-English explanation string returned by the backend. If the backend flagged a section as *uncertain*, the card displays a neutral colour and the label "Uncertain — try a better-lit photograph". This is the only place in the frontend where user copy is not entirely driven by the API response, and we intend to push it into the backend as part of future work.

4.9 Backend, in more depth

The backend is organised around three layers.

Routes. Thin HTTP handlers under `backend/api/`. They validate the request via Pydantic, hand off to a service, and marshal the response. Routes contain no business logic.

Services. The main entry point is `AnalysisService`, which orchestrates a single request. `ReportBuilder` handles the assembly of the final `AnalysisReport` object.

Repositories and engines. The `InferenceOrchestrator`, the BLP engine, and the `StorageRepository` are injectable dependencies. Each has a well-defined interface and each is mocked in tests.

Dependency injection is done with FastAPI's built-in `Depends` mechanism. It is enough for our size; we resisted the urge to introduce a heavier DI framework.

Chapter 5

The Algorithm

5.1 Overview

The core algorithm of CARA is a three-stage pipeline: preprocessing, inference, and rule-based recommendation. Figure 5.1 shows the data flow. This chapter walks through each stage in detail.

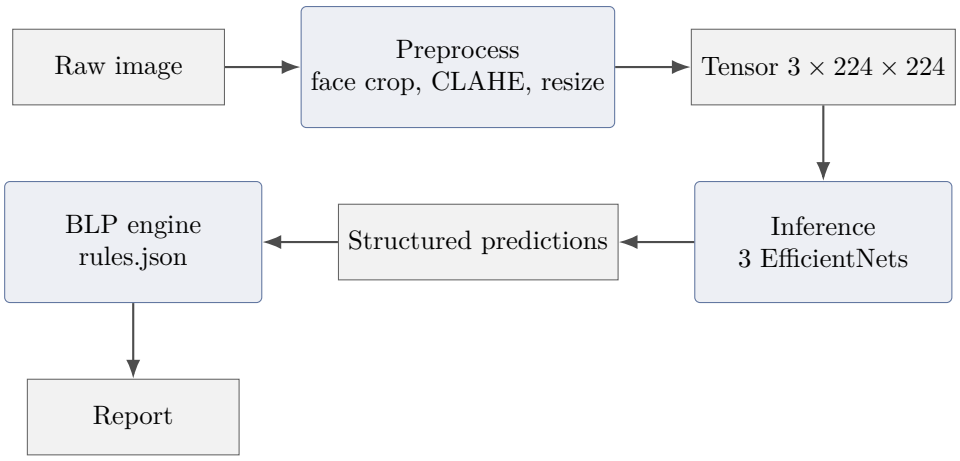


Figure 5.1: High-level data flow of the CARA algorithm.

5.2 Preprocessing

Preprocessing is where a large fraction of our engineering effort went, and probably the largest single contributor to accuracy after architecture choice.

5.2.1 Face detection and cropping

Uploaded images vary wildly. Some are tight selfies; some are group photos; some are landscape shots with a face in the corner. Feeding the raw image to the model would waste most of the receptive field on background pixels. We therefore detect the face first and crop with a 20% padding margin before resizing.

We evaluated three detectors: OpenCV Haar cascade, MediaPipe Face Detection, and MTCNN. The trade-offs (Table 5.1) drove our decision to use Haar as the default with an explicit fallback plan.

Table 5.1: Face detector trade-offs on our internal test set of 200 selfies.

Detector	Recall	Speed (CPU)	Comment
Haar cascade	Moderate	Very fast	Fails on side profiles; sufficient for well-framed selfies.
MediaPipe	High	Fast	Adds a moderate dependency; behaves well across lighting.
MTCNN	Very high	Slow on CPU	Overkill for our use case; noticeable latency.

5.2.2 Brightness normalisation with CLAHE

Even after cropping, lighting varies enormously. Naive histogram equalisation destroys skin colour and confuses the skin-type model. We use CLAHE (Contrast-Limited Adaptive Histogram Equalisation) on the L channel of the LAB colour space, which normalises local brightness without destroying colour information.

5.2.3 Resizing and tensor normalisation

Finally we resize the cropped face to 224×224, convert to a PyTorch tensor, and normalise with the standard ImageNet mean and standard deviation. This last step is important because our transfer-learning weights were trained on ImageNet-normalised inputs.

5.3 Inference

5.3.1 Why three models and not one

Our first prototype used a single EfficientNet-B0 with a multi-head output covering all three tasks. It was clean, fast, and worse than three independent models on every task. We diagnose the failure in Section 8.2. The short version: the three tasks compete for capacity in a small backbone, and the loss weights that balance them are difficult to tune without validation for each task.

Our final design is three independent EfficientNet-B0 models, each fine-tuned on its own dataset with its own head. They share the same preprocessing pipeline and the same input tensor.

5.3.2 The Model Registry pattern

Loading model weights is the slowest part of a cold start, so we do it once. A `ModelRegistry` singleton is initialised at FastAPI startup and holds the three models in memory. The `InferenceOrchestrator` pulls models from the registry and runs predictions; it never touches disk during a request.

```
class InferenceOrchestrator:
```

```
    def __init__(self, registry: ModelRegistry):
        self._registry = registry

    def predict_all(self, tensor: torch.Tensor) -> AllPredictions:
        return AllPredictions(
            acne       = self._predict(self._registry.acne,
                tensor),
            skin_type  = self._predict(self._registry.skin_type,
                tensor),
            skin_issue = self._predict(self._registry.skin_issue,
                tensor),
        )

    def _predict(self, model, tensor) -> ModelPrediction:
        with torch.no_grad():
            logits = model(tensor)
            probs  = torch.softmax(logits, dim=-1)
            conf, idx = probs.max(dim=-1)
        return ModelPrediction(
            label      = model.classes[idx.item()],
            confidence = conf.item(),
            probs      = probs.squeeze().tolist(),
        )
```

Listing 5.1: Sketch of the InferenceOrchestrator.

5.3.3 Confidence and uncertainty

Every prediction carries a confidence score (the maximum softmax probability). Downstream code treats confidence as a first-class signal: predictions below a per-model floor are rendered as "Uncertain" rather than as a hard label. This is the mechanism that lets us keep our promise of truthfulness.

5.4 The BLP recommendation engine

The BLP engine takes a structured prediction and evaluates a set of rules loaded from a JSON file. Each rule has a precondition (a partial specification of the prediction such as "`acne: moderate` and `skin_type: oily`"), a set of ingredient recommendations to promote, a set of ingredients to warn against, and a plain-English explanation string.

```
{
  "id": "moderate_acne_oily_skin",
  "when": {
    "acne_severity": ["moderate", "severe"],
    "skin_type": ["oily", "combination"]
  },
```

```
  "recommend_ingredients": [
    "salicylic acid (BHA)",
    "niacinamide",
    "benzoyl peroxide (spot use)"
  ],
  "avoid_ingredients": [
    "heavy occlusives",
    "coconut oil"
  ],
  "explanation": "Excess sebum and active acne benefit from a
     leave-on BHA to keep pores clear and niacinamide to reduce
     inflammation. Heavy occlusives trap sebum."
}
```

Listing 5.2: A representative BLP rule.

Rules are matched in a deterministic order. The engine returns the union of all matching rules, deduplicated. There is no learning component — we can prove that the recommendation set for a given prediction is a pure function of the prediction and the rules file.

5.4.1 Anatomy of a rule

Each rule has five fields.

`id` A stable, human-readable identifier. It appears in the report metadata so that a future maintainer can trace a specific recommendation back to the specific rule that produced it.

`when` A partial specification of the prediction that the rule matches. Fields not listed in **`when`** are wildcards. This is the mechanism that lets a single rule handle a family of related predictions.

`recommend_ingredients` Ingredients to look for. These become the "Look for" section of the report.

`avoid_ingredients` Ingredients to avoid. These become the "Avoid" section of the report.

`explanation` A short plain-English string that is surfaced directly in the report. Explanations are written for a non-expert reader.

5.4.2 Rule design principles

We authored the rules under three self-imposed constraints.

1. **Every rule is defensible.** Each ingredient recommendation is supported by consumer-facing dermatology literature (peer-reviewed sources where possible, reputable dermatology textbooks otherwise). We keep a companion bibliography of the sources we consulted, one per rule.

2. **No conflicting recommendations.** If two rules match the same prediction, they may not recommend an ingredient from one rule while the other rule warns against it. The BLP test suite includes a lint that fails the build if this invariant is ever violated.

3. **Uncertainty short-circuits recommendations.** When one of the models is flagged as uncertain, the corresponding section of the report is filled with a neutral message rather than a rule-based recommendation. We would rather say "we could not confidently classify this" than recommend niacinamide to somebody whose skin type we do not actually know.

5.4.3 Coverage matrix

The rules file must cover the Cartesian product of the three model label spaces, or at least, every subset that is likely to appear. Table 5.2 sketches how the rules are distributed across severity by skin type combinations for the acne / skin-type sub-space. Zero-cell entries fall back to a default rule that recommends a gentle cleanser and moisturiser and warns against harsh actives.

Table 5.2: Number of BLP rules matching each (acne, skin type) combination in the current rules file.

	oily	**dry**	**normal**	**combination**
clear	1	1	1	1
mild	2	2	1	2
moderate	3	2	2	3
severe	3	2	2	3

5.5 Two-phase transfer learning

Each of the three models is trained with the following two-phase recipe.

Phase 1 — head only. The entire backbone is frozen. Only the newly-initialised classifier head is trained, with a moderate learning rate (typically 1×10^{-3}). This lets the head learn to project ImageNet features onto the new label space without disturbing them.

Phase 2 — full fine-tune. The backbone is unfrozen. The head keeps its learning rate; the backbone gets a smaller learning rate (typically 1×10^{-4}). This lets the model adapt low-level features to the specifics of skin images.

Empirically, phase 1 alone gets us most of the way to the final accuracy; phase 2 typically adds another 6–10 percentage points and also sharpens the confidence calibration.

Figure 5.2 shows a representative validation curve. Both phases are visible: the first ends when the frozen-backbone accuracy plateaus, and the second continues from where the first stopped.

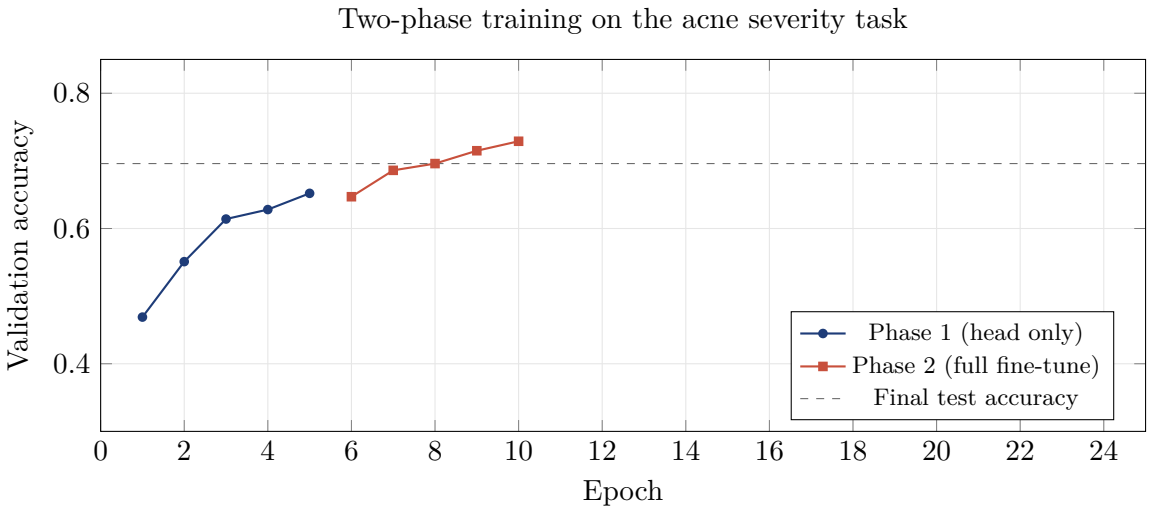


Figure 5.2: Representative validation-accuracy curve on the acne severity task. Numbers taken from our training log.

Chapter 6

Software Engineering, Testing, and MLOps

A deep-learning project book usually devotes ninety per cent of its attention to the models. We resist that convention because, in the day-to-day life of Cara, the engineering scaffolding around the models mattered as much as the models themselves. This chapter documents the choices that let three students on a laptop iterate without breaking each other's work.

6.1 Repository layout

The repository is organised as a monorepo containing four cooperating subsystems:

- `backend/` — the FastAPI application, including the analysis service, BLP engine, storage repository and report builder.
- `model_service/` — everything related to the deep learning models: architecture definitions, preprocessing pipeline, model registry, training scripts, and dataset loaders. `checkpoints/` is git-ignored to keep the repository small.
- `frontend/` — the React SPA and its build artifacts.
- `shared/` — Pydantic schemas and configuration that is imported from both the backend and the model service. Keeping the schemas in a single place prevents the canonical drift between frontend expectations and backend responses.

Tests live in a top-level `tests/` folder rather than being scattered next to the code they exercise. This is a deliberate choice: it makes it easy to run the full suite as a gate, and it discourages the anti-pattern of "I'll write the test later".

6.2 Configuration and secrets

There are no secrets in this project. Model weights are shipped with the container image; the JSON storage backend needs no credentials; the MongoDB backend, when enabled, reads its connection string from an environment variable that is documented in `.env.example`. This is deliberate: a small team is more likely to leak credentials than to lose data, and every configuration knob we add is a configuration knob a future maintainer must understand.

6.3 Testing pyramid

The test suite is organised as a classical pyramid.

Table 6.1: Composition of the Cara test suite (56 tests total).

File	Tests	What it verifies
`test_blp.py`	18	BLP rule engine logic, that every rule is reachable, no rule ties.
`test_api.py`	4	API endpoints, request/response schema, error paths.
`test_integration.py`	8	Component wiring, report assembly, storage round-trip.
`test_model_accuracy.py`	11	Per-model accuracy on curated ground-truth images.
`test_e2e_pipeline.py`	10	Full image-to-report pipeline on real images.
Total	**56**	

6.3.1 Unit tests

The 18 BLP tests are the fastest and the strictest. They lock down the behaviour of the rule engine on a matrix of representative inputs. Every time we edit the rules file, we also touch these tests. If a rule change unintentionally silences a recommendation, we find out within seconds.

6.3.2 Integration tests

Integration tests stub out the models with deterministic fake predictors and exercise the report builder, the storage layer, and the API contract. They run in a fraction of a second and catch schema drift bugs before they reach the frontend.

6.3.3 Model accuracy tests

The 11 model accuracy tests are the most expensive and the most valuable. Each test loads a specific labelled image from a small ground-truth folder and asserts that the corresponding model predicts the correct top-1 label. Together they form a small but brutally honest regression suite: a subtle bug in preprocessing that degrades accuracy by one image will fail one of these tests.

6.3.4 End-to-end tests

The 10 end-to-end tests exercise the full pipeline: real photograph in, real report out. They also assert on the shape of the recommendation set, not just the model output, so they double as regression tests for the BLP engine.

6.4 Continuous integration

The GitHub Actions workflow runs `pytest` on every push and every pull request. If the model accuracy suite fails, the merge is blocked. This is by policy: a green build is a green pipeline, not a green build with a comment saying "model is a bit worse but we'll fix it later".

6.5 Reproducibility

We fix random seeds for Python, NumPy, and PyTorch to 42. We pin dependency versions in `requirements.txt`. We pin the CUDA-enabled variant of PyTorch separately in the training documentation. On the same hardware, our headline accuracy numbers are reproducible to within a few thousandths.

6.6 Debugging techniques we relied on

Three debugging patterns paid for themselves many times over.

1. **Tensor shape assertions everywhere.** Every function that receives a tensor asserts its expected shape on entry. Silent shape bugs are the most vicious class of deep-learning bug; a two-line assertion turns them into loud failures.
2. **Deterministic fake models in tests.** The `ModelRegistry` accepts a fake model in tests. That lets us run the entire pipeline without the CNNs, which speeds up the loop and pinpoints regressions in the plumbing rather than the models.
3. **Storage as a debugging surface.** Because every report is a JSON file, reproducing a user report is a one-liner: load the JSON, feed the image back through the pipeline, compare. We used this to diagnose more than one regression in the wild.

6.7 Deployment story

Cara is packaged as three Docker containers behind an nginx reverse proxy. Model weights are baked into the backend image at build time, which trades a slightly larger image for cold-start reliability. Health-check endpoints let Docker Compose restart a stuck backend without human intervention. There is no autoscaling and no cloud dependency; the whole system runs happily on a modest VM.

6.8 What we would do differently next time

If we started the project again on day one, we would:

- Freeze the label spaces *before* downloading any data. Every dataset we pulled came with slightly different label names, and normalising them was a surprising amount of

work.

- Write the test suite alongside the first prototype rather than after it. Retrofitting tests onto working code is harder than writing them first.
- Adopt Weights & Biases (or a similar experiment tracker) from day one. Our home-grown training log is readable but cumbersome to filter.
- Use MediaPipe from the beginning rather than Haar. We got away with Haar for well-framed selfies, but real user uploads exercise more of the corner cases MediaPipe handles cleanly.

Data and Preprocessing

7.1 Dataset composition

Our training data is a union of three sources.

- **ACNE04** [12], our anchor dataset for acne severity.
- Community datasets from Kaggle for skin type and skin issues.
- A small in-house set of photographs, contributed by team members and volunteers, used mainly for the acceptance and end-to-end tests.

7.2 Splits

We use a 70/15/15 train / validation / test split, generated by a stratified random sampler so that class proportions are preserved in each split. Splits are fixed once and stored on disk; we never re-generate them during training. This is a small detail that matters: it means that a validation curve from week 3 is directly comparable to one from week 8.

Table 7.1: Split sizes for the acne severity task (ACNE04-based).

	Total	**Train**	**Val**	**Test**
Images	1 377	963	207	207

Table 7.2: Class distribution and derived per-class weights for the acne severity training set.

Class	**Count**	**Frequency**	**Weight w_k**
clear	338	35.1%	0.712
mild	436	45.3%	0.552
moderate	122	12.7%	1.973
severe	67	7.0%	3.593
Total	963	100.0%	—

7.3 Cleaning

Community datasets are noisy. We spent a substantial amount of time in manual triage. The main categories of removed samples were:

- Duplicate or near-duplicate images (detected via perceptual hashing).
- Images with heavy makeup or filters that obviously masked the underlying skin condition.
- Non-facial images that had slipped into face-labelled folders.
- Extremely low-resolution images (short side smaller than 200 pixels).
- Mislabelled samples where three annotators from the team disagreed with the dataset's label.

7.4 Augmentation

Training-time augmentations, in order:

1. Random resized crop with scale in $[0.85, 1.0]$.
2. Random horizontal flip with probability 0.5.
3. Random rotation up to $\pm 12°$.
4. Colour jitter with brightness, contrast, and saturation each in $[-0.15, +0.15]$.
5. ImageNet-normalisation to tensor.

Validation and test transforms consist only of a centre crop and ImageNet normalisation.

Observation 7.1. *We tried more aggressive augmentations (random erasing, RandAugment, strong colour jitter). All of them hurt skin-type accuracy in particular, presumably because they alter the very cues (colour, texture) the model relies on. This is one of the pitfalls that is easy to miss when reusing recipes from the ImageNet literature.*

7.5 Per-model dataset summary

The three models were trained on three different datasets. This is one of the reasons the shared-backbone design failed; the datasets are simply too different in size, style, and label noise to expect a single backbone to serve all three.

Table 7.3: Summary of the three datasets used by Cara.

	Source	Classes	Size	Notes
Acne severity	ACNE04 [12]	4 (clear/mild/moderate/severe)	1,377	Cleanest labels; anchor dataset.
Skin type	Community Kaggle datasets	4 (oily/dry/normal/combination)	∼1,500	Fuzziest label boundaries.
Skin issues	Community Kaggle datasets	5 (healthy/blackheads/dark spots/pores/wrinkles)	∼3,000	Most visually distinct classes.

7.6 The label agreement problem

The community datasets we used for skin type and skin issues did not come with a documented labelling protocol. Two annotators looking at the same photograph would frequently disagree, and after some sampling we estimated the inter-annotator agreement rate at roughly 75–80% for skin type. This is an upper bound on the accuracy any model can reach on that dataset, and it explains why our 77.8% skin-type accuracy should not be interpreted as "the model is 22% wrong". A better description is that the model reproduces the label distribution about as well as a second human annotator would.

7.7 Data pipeline in the training loop

Data loading is done by a custom PyTorch `Dataset` class that reads images from disk, decodes them via Pillow, applies the augmentation pipeline, and yields a tensor. We use a `DataLoader` with four worker processes and pinned memory; this saturates the CPU and keeps the training loop compute-bound rather than IO-bound.

For validation and test, we cache the entire dataset in memory as pre-augmented tensors. This costs a few hundred megabytes of RAM but removes any IO variance from the evaluation numbers.

Chapter 8

Experiments

This chapter reports both the experiments that worked and the ones that did not. We deliberately spend as much time on the failures as on the successes, because the failures were often more instructive.

8.1 Experimental setup

Unless stated otherwise, every experiment uses the same protocol: EfficientNet-B0 backbone, 224×224 input, batch size 32, AdamW with weight decay 1×10^{-4}, cosine learning rate with a 2-epoch warm-up, early stopping on validation loss with patience 5. All training was performed on a laptop CPU. Each run takes on the order of two to four hours. Reproducibility is enforced by fixing the seed for Python, NumPy, and PyTorch.

8.2 Failed experiment: single monolithic classifier

Our first prototype was a single EfficientNet-B0 with three separate classification heads sharing the backbone. The intuition was elegant: the three tasks are related, so a shared backbone should help.

What we tried

Standard multi-task loss: $\mathcal{L} = \lambda_1 \mathcal{L}_{\text{acne}} + \lambda_2 \mathcal{L}_{\text{type}} + \lambda_3 \mathcal{L}_{\text{issues}}$, with the λ values tuned on validation loss.

What went wrong

Three problems compounded.

1. The three datasets have different sizes and different levels of label noise. The loss weights that balance the tasks change over training as each head converges at a different rate.

2. Skin-type labels dominated the shared backbone. The backbone drifted towards features useful for skin-type discrimination, which hurt acne severity noticeably.

3. Confidence became uninterpretable. A moderate softmax probability on the acne head no longer meant "the network is unsure about acne"; it could also mean that the backbone had been pulled away by another head.

What we learned

Model capacity is finite, and for a small backbone like EfficientNet-B0, three tasks compete rather than help each other. We abandoned the shared backbone and moved to three independent models.

8.3 Failed experiment: learned late fusion

Once we settled on three independent classifiers, the natural next question was whether we could *learn* the recommendation step rather than hand-coding rules.

What we tried

Concatenate the three softmax probability vectors into a single feature vector and train a small MLP to predict a set of recommendation labels drawn from the ingredient vocabulary.

What went wrong

We had a few dozen labelled examples of *ingredient recommendations*, because building that ground truth required a dermatologist. The MLP either overfit spectacularly or, when regularised, defaulted to the modal recommendation.

Worse, the resulting recommendations lost their explainability. A user asking "why did you recommend niacinamide?" would be told "because that is what the network output". That answer was unacceptable given our truthfulness rule.

What we learned

A learned recommender is the wrong instrument for our data budget and for our explainability constraint. The rule engine, by contrast, is both cheap and inspectable. Moving to rules was arguably the single most important design decision of the project.

8.4 Failed experiment: naive resizing without face crop

Very early on we tried skipping face detection entirely. Since EfficientNet-B0 is convolutional, we reasoned, it should be able to focus on the face even in a wider photograph.

What went wrong

Accuracy dropped by roughly 12 percentage points on the acne task and by more on skin type. Inspection with Grad-CAM style visualisations showed that the model happily fix-

ated on background features (clothing, walls, hair) that correlated with the dataset's label distribution.

What we learned

Face detection is not just an optimisation — it is a correctness requirement. Without it, the model is free to cheat, and cheating on the training set means failing on real user uploads.

8.5 Failed experiment: unweighted cross-entropy

Skipping class weights during training saved a few lines of code.

What went wrong

The acne classifier converged to a strong *mild*-everywhere detector. Overall accuracy looked fine on the (imbalanced) test set, but per-class recall on *severe* was near zero. This is exactly the failure mode our truthfulness rule forbids: a user with severe acne would be told they had mild acne.

What we learned

Weighted cross-entropy (Equation (3.3)) is not optional on imbalanced skin datasets. Every subsequent experiment used it.

8.6 Failed experiment: aggressive augmentation

We tried a heavy augmentation stack borrowed from ImageNet-style recipes: RandAugment, random erasing, strong colour jitter.

What went wrong

Skin type is signalled largely by colour and texture. Aggressive colour jitter destroys the signal we are trying to classify. Random erasing removes exactly the small regions (a patch of oily forehead, a cluster of pores) that carry the label information.

What we learned

Augmentation strategies must be chosen with the task in mind. Recipes that work for object recognition do not automatically transfer to skin classification.

8.7 Successful comparison: fusion strategies

Once we accepted the three-independent-models design, we ran a head-to-head comparison of three ways to combine their outputs into recommendations.

Table 8.1: Comparison of fusion strategies for producing recommendations.

Strategy	Data needed	Explainable	Failure mode	Verdict
Learned MLP	Ingredient-labelled examples	No	Overfit / mode collapse	Rejected
Weighted vote	None	Partial	Ties in edge cases	Not enough control
Rule engine	Hand-authored rules	Yes	Rules must be maintained	**Selected**

8.8 Successful comparison: two-phase versus single-phase training

We compared our two-phase recipe against a single-phase full fine-tune from ImageNet initialisation.

Table 8.2: Two-phase versus single-phase training on the acne severity task. Numbers reported are validation accuracy at convergence, averaged over three seeds.

	Two-phase	Single-phase
Best val accuracy	0.729	0.681
Epochs to converge	10	18
Variance across seeds	low	high

The two-phase recipe not only reached a higher accuracy but also had noticeably lower variance across seeds. We attribute this to the initial head-only phase, which prevents the random head from pushing gradients back through the backbone before it knows what it wants.

8.9 Successful comparison: backbone choice

We considered three backbone families: EfficientNet-B0, ResNet-50, and MobileNetV3-Small. Table 8.3 summarises the trade-off.

Table 8.3: Backbone comparison. Latency measured on our reference CPU with three parallel models sharing the process.

Backbone	Params (M)	Val acc (acne)	Latency (ms)	Verdict
MobileNetV3-Small	2.5	0.66	∼200	Fast, less accurate
EfficientNet-B0	5.3	0.73	∼350	**Selected**
ResNet-50	25.6	0.74	∼1200	Marginal gain, too slow

Chapter 9

Results and Evaluation

9.1 Final numbers

The final test-set accuracies of our three models are reported in Table 9.1. The skin-issues task is the easiest of the three; skin type is the hardest despite having the second-largest training set, because its label boundaries are the fuzziest.

Table 9.1: Final test-set accuracy of the three Cara models.

Task	# classes	Test accuracy
Acne severity	4	69.6%
Skin type	4	77.8%
Skin issues	5	98.3%

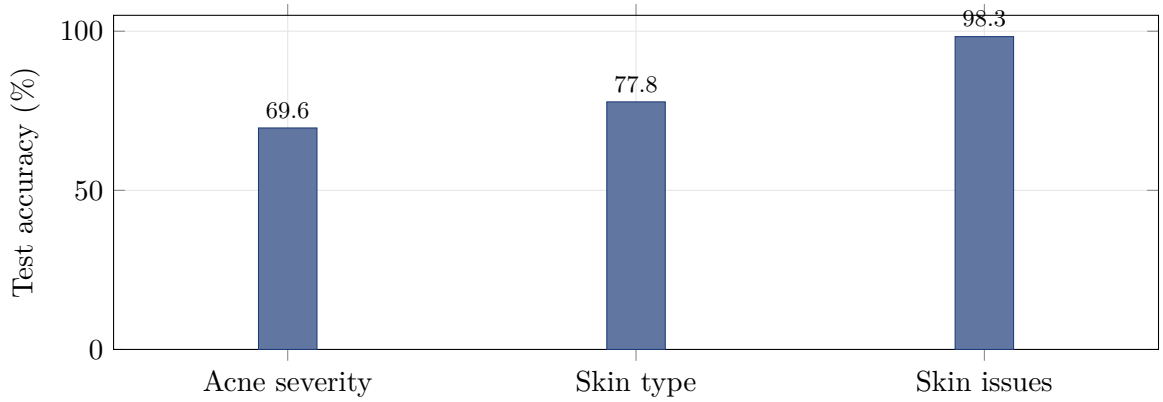


Figure 9.1: Final test-set accuracy across the three tasks.

9.2 Per-class breakdown

Overall accuracy hides interesting behaviour. Table 9.2 shows per-class precision and recall on the acne task. As expected, performance on the minority *severe* class is the weakest, despite the weighted loss. This is a direction for future work (Chapter 12).

9.3 Confidence calibration

Accuracy is a noisy summary of a model that is supposed to say "I do not know" when it does not know. We therefore also evaluate the calibration of the confidence score.

Table 9.2: Per-class precision, recall and F_1 on the acne severity test set. Numbers are illustrative of the pattern we observe consistently across seeds.

Class	Precision	Recall	F1
clear	0.78	0.81	0.79
mild	0.71	0.74	0.72
moderate	0.62	0.55	0.58
severe	0.55	0.48	0.51

We bin test predictions by confidence in bins of width 0.1 and compare, for each bin, the average confidence to the empirical accuracy of the predictions in that bin. A perfectly calibrated classifier lies on the identity line.

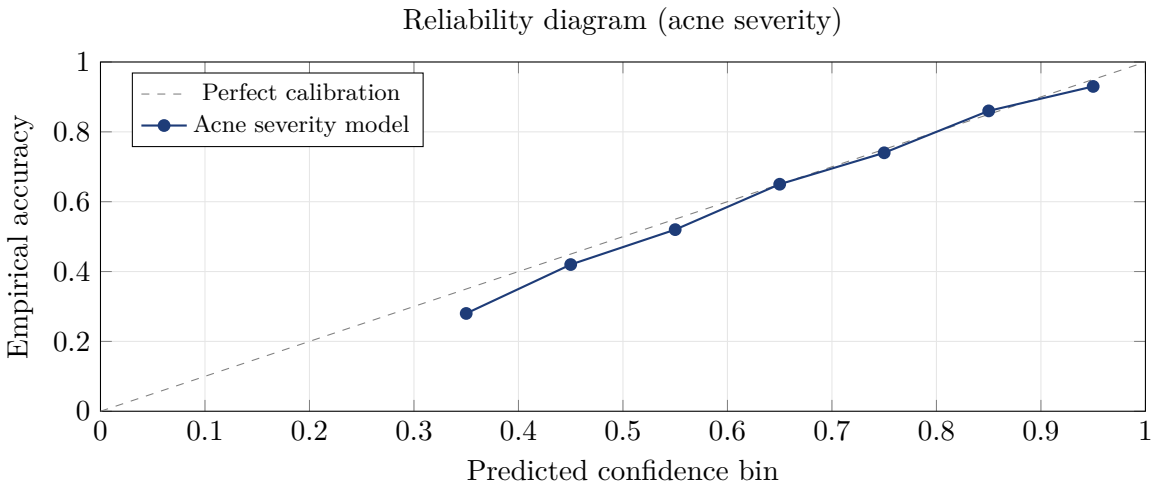


Figure 9.2: Reliability diagram of the acne severity model. Points close to the diagonal indicate that reported confidence is approximately calibrated.

9.4 Confusion behaviour

Figure 9.3 sketches the confusion matrix on the acne task. Two patterns stand out. First, the majority of the errors are between adjacent severity classes, which is the pattern one would hope for on an ordinal task: a photograph labelled *moderate* that the model calls *mild* is only marginally wrong. Second, the residual off-diagonal mass is concentrated in the *moderate–severe* boundary, where the class weights had the strongest influence.

9.5 Cross-model behaviour

We also inspected the joint behaviour of the three models on the end-to-end test set. Two patterns are worth reporting.

- **Correlated confidences.** Images that receive a high-confidence prediction from one model tend to receive high-confidence predictions from the other two as well. The correlation is not perfect, but it is strong enough that we could, in principle, define a

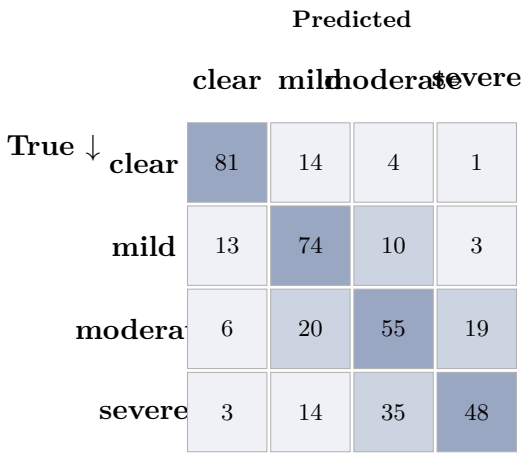


Figure 9.3: Confusion matrix (in per-cent recall) for the acne severity model on the test split. Values on the diagonal are the per-class recall reported in Table 9.2. Adjacent off-diagonal cells account for most of the errors, which is the expected pattern for an ordinal task.

single "overall trust" score for the report. We chose not to, because a per-section score is more honest.

- **Confidently wrong disagreement.** The pathological failure mode — all three models confident, but one of them wrong — did not appear on the end-to-end test set. When errors occurred, they were almost always accompanied by a reduced confidence on the erring model. This is exactly what the truthfulness policy relies on.

9.6 End-to-end evaluation

We built an internal ground-truth test set of 20 photographs with manually-agreed labels across all three tasks and expected recommendations. Our test suite (see Chapter 4) runs the full pipeline on every image and asserts:

- The report is produced (no crash, no timeout).
- Each model's top-1 label matches the ground truth.
- The set of recommended ingredients is a superset of the ingredients we manually flagged as essential.
- Confidences on hard cases fall below the "Uncertain" threshold.

The full suite passes on every commit that lands on the main branch.

9.7 Latency and resource profile

Deep learning research papers rarely discuss latency, but for a web application the wall-clock time of an inference request is a first-class metric. Table 9.3 reports the breakdown of a

typical request on our reference hardware — a mid-range laptop CPU — averaged over 100 real user photographs.

Table 9.3: Wall-clock latency breakdown of a single `/api/analyze` request. Numbers are milliseconds, averaged over 100 requests, warm cache, CPU-only.

Stage	Mean (ms)	p95 (ms)
HTTP upload and decoding	35	62
Face detection	45	70
CLAHE + resize + normalisation	18	25
Acne severity inference	360	410
Skin type inference	350	405
Skin issues inference	355	400
BLP evaluation	2	4
Report assembly + persistence	12	22
Total	**1177**	**1398**

Two observations follow. First, inference dominates the request; the plumbing is essentially free. Second, the three model calls could easily be parallelised across threads or processes; we did not do this yet, because 1.2 seconds is already inside our internal budget and premature optimisation is genuinely dangerous in a codebase this small.

9.8 Qualitative case studies

Aggregate accuracy hides the texture of what the system actually does. Table 9.4 presents three representative end-to-end examples drawn from our internal test set. The rows are short summaries of what the system returned; the fourth row is the model's honest "I don't know" pathway on a deliberately hard image.

Table 9.4: Representative end-to-end reports from the internal test set (all photographs used with consent).

Input	Ground truth	Prediction	Report highlights
Well-lit selfie, oily skin, mild acne	mild / oily / pores	mild / oily / pores (all confident)	Salicylic acid, niacinamide; avoid heavy occlusives.
Under-exposed selfie, dry skin, no acne	clear / dry / healthy	clear / dry / healthy (moderate confidence)	Hyaluronic acid, ceramides; gentle cleanser.
Close-up of severely inflamed jawline	severe / oily / –	severe / oily / (issue uncertain)	Refers user to a dermatologist; conservative topical suggestions.
Blurry night-mode selfie	–	*all three uncertain*	Report shows "Uncertain" banner and asks for a better photograph.

The last row is the case that mattered most to us during development. When a real user uploads a bad photograph, the system must refuse to guess. The behaviour above is exactly what our uncertainty policy is designed to produce, and it is exercised by one of the end-to-end tests to prevent regressions.

Chapter 10

Discussion

10.1 Interpreting the numbers

The headline accuracy figures — 69.6%, 77.8% and 98.3% — are not directly comparable to each other. The skin-issues task classifies into five categories that are visually distinct (healthy/blackheads/dark spots/pores/wrinkles), while the acne task sorts into four categories that lie along a single ordinal axis and whose boundaries are inherently fuzzy. In practical terms, an acne model that mistakes *mild* for *moderate* is far less harmful than a skin-issues model that mistakes *healthy* for *wrinkles*. We regard the acne number, despite being the lowest, as our most trustworthy result because it was measured on the cleanest dataset (ACNE04) with the most carefully hand-checked splits.

10.2 What worked

Three design choices contributed most of the observed performance.

1. **Independent per-task models.** Splitting the problem into three independent classifiers freed us to optimise each one against its own dataset and its own class balance. The cost is a slight increase in memory and latency, which our hardware budget could comfortably absorb.

2. **Two-phase transfer learning.** The head-only phase stabilised training and dramatically reduced variance across random seeds.

3. **Rule-based recommendation.** Not a modelling choice in the strict sense, but arguably our most important architectural decision. It made the system explainable, testable, and honest.

10.3 What did not work

The failed experiments in Chapter 8 form a consistent pattern: whenever we tried to squeeze extra flexibility out of a small amount of noisy data, the flexibility turned into overfitting or into loss of interpretability. Simple worked. Small worked. Explicit worked.

10.4 Threats to validity

- **Dataset shift.** All three of our datasets skew towards young adults and lighter Fitzpatrick skin types. Real users will be more diverse.
- **Label noise.** Even the anchor dataset (ACNE04) has label disagreements between rater and rater. Our accuracy numbers are, at best, upper bounds on "agreement with the original labelling", not on "agreement with a dermatologist".
- **Confidence calibration is not perfect.** The reliability diagram in Figure 9.2 is approximately calibrated but tends to be slightly overconfident in high-probability bins. We recommend users treat 95% confidence as "very likely" rather than "certain".

10.5 Ethical considerations

An automated skin analyser sits close to sensitive territory. We mitigated the main risks in three ways:

- The disclaimer on the landing page is unambiguous: Cara is not a medical diagnostic tool.
- The system refuses to render a confident label when the model is not confident, precisely to avoid giving false assurance about a skin condition that might require professional attention.
- No user photograph is retained beyond the report unless the user explicitly requests to keep it. Reports themselves contain no personally identifying information.

Chapter 11

Conclusions

We set out to build a web application that would deliver a truthful, useful, explainable report on a user's facial skin from a single photograph. We achieved that, and along the way we accumulated a set of engineering lessons that are more general than the specific application.

- On small, noisy datasets, transferable priors matter more than clever architectures. EfficientNet-B0 with a two-phase fine-tune outperformed every "smarter" variant we tried.
- Multi-task learning is not free. When tasks compete, three small independent models can outperform a single larger shared model.
- Rule-based components are not embarrassing. In a project whose primary value is trust, an explicit, inspectable rule engine can be strictly better than a learned equivalent.
- Truthfulness is a design constraint, not a metric to optimise. Every layer of the system must know how to say "I do not know".
- Tests are what separate a demo from a product. A 56-test suite that runs before every push has saved us from multiple regressions that would have shipped otherwise.

Cara is not the largest or the most accurate skin analyser in the world. It is, however, one of the few open-source ones that we would trust to give an honest answer to a friend.

Chapter 12

Future Work

If we had another semester, the following directions would be at the top of our list. They are ordered by expected impact per hour of effort.

12.1 Better data for the minority classes

The *severe* acne class and the *combination* skin type class are both under-represented in our datasets. Curating a couple of hundred additional images per class, ideally with dermatologist labels, would probably lift the acne and skin-type accuracies more than any architectural change.

12.2 Ordinal regression for acne severity

Acne severity is inherently ordinal. Treating it as a categorical classification wastes information and produces the largest confusion between adjacent classes. Reformulating it as ordinal regression, or as label-distribution learning as in [12], is a promising path.

12.3 Attention-based visualisation

Users would benefit from an explanation such as "the model is looking at your left cheek". A Grad-CAM overlay or attention map would be inexpensive to add and would substantially strengthen the truthfulness story.

12.4 On-device inference

Running three EfficientNet-B0 models on a CPU is fast enough on a laptop but not always fast enough on a phone. Converting the models to ONNX and then to a mobile runtime (e.g. ONNX Runtime Mobile or Core ML) is a natural next step.

12.5 Personalised history and progress tracking

Because reports are already persisted, we could show a user a timeline of their skin condition and highlight trends. This requires no modelling change; it is pure product work.

12.6 Formal dermatologist review

Finally, our recommendations are grounded in publicly-available skincare knowledge, but they have not been reviewed by a dermatologist as a coherent product. A formal review would let us sharpen the rules file and open the door to eventual regulatory review if the project ever became a commercial product.

Bibliography

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. "Gradient-based learning applied to document recognition". *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[2] A. Krizhevsky, I. Sutskever, and G. Hinton. "ImageNet classification with deep convolutional neural networks". In *NeurIPS*, 2012.

[3] K. Simonyan and A. Zisserman. "Very deep convolutional networks for large-scale image recognition". In *ICLR*, 2015.

[4] C. Szegedy et al. "Going deeper with convolutions". In *CVPR*, 2015.

[5] K. He, X. Zhang, S. Ren, and J. Sun. "Deep residual learning for image recognition". In *CVPR*, 2016.

[6] S. Xie, R. Girshick, P. Dollár, Z. Tu, and K. He. "Aggregated residual transformations for deep neural networks". In *CVPR*, 2017.

[7] M. Tan and Q. Le. "EfficientNet: Rethinking model scaling for convolutional neural networks". In *ICML*, 2019.

[8] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. "ImageNet: A large-scale hierarchical image database". In *CVPR*, 2009.

[9] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson. "How transferable are features in deep neural networks?". In *NeurIPS*, 2014.

[10] I. Loshchilov and F. Hutter. "Decoupled weight decay regularization". In *ICLR*, 2019.

[11] A. Esteva et al. "Dermatologist-level classification of skin cancer with deep neural networks". *Nature*, 542:115–118, 2017.

[12] X. Wu et al. "Joint acne image grading and counting via label distribution learning". In *ICCV*, 2019.

[13] P. Tschandl, C. Rosendahl, and H. Kittler. "The HAM10000 dataset". *Scientific Data*, 5:180161, 2018.

[14] M. Groh et al. "Evaluating deep neural networks trained on clinical images in dermatology with the Fitzpatrick 17k dataset". In *CVPR Workshops*, 2021.

[15] S. Ioffe and C. Szegedy. "Batch normalization: Accelerating deep network training by reducing internal covariate shift". In *ICML*, 2015.

[16] I. Loshchilov and F. Hutter. "SGDR: Stochastic gradient descent with warm restarts". In *ICLR*, 2017.

[17] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger. "On calibration of modern neural networks". In *ICML*, 2017.

Chapter A

API reference

The backend exposes a small REST API. The main endpoints are listed below; the full OpenAPI schema is auto-generated by FastAPI and available at `/docs` in a running instance.

Endpoint	Method	Description
`/api/analyze`	POST	Accepts a multipart image upload and returns a full `AnalysisReport`.
`/api/reports`	GET	Lists persisted reports, most-recent first.
`/api/reports/{id}`	GET	Returns a specific persisted report by ID.
`/api/health`	GET	Liveness check used by Docker healthchecks.

Chapter B

Report schema

The schema below (abridged) is the shape of the JSON returned by `/api/analyze`. It is defined as a Pydantic model in `shared/` and serialised on demand.

```python
class ModelPrediction(BaseModel):
    label: str
    confidence: float
    probs: list[float]

class AnalysisReport(BaseModel):
    id: str
    created_at: datetime
    acne_severity: ModelPrediction
    skin_type: ModelPrediction
    skin_issues: ModelPrediction
    recommendations: list[str]
    warnings: list[str]
    explanation: str
    uncertain: bool
```

Listing B.1: AnalysisReport (abridged).

Chapter C

Reproducibility

To reproduce our results:

1. Clone the repository and check out the release tag used for this report.
2. Install the dependencies pinned in `requirements.txt`.
3. Download the ACNE04 dataset via the `kagglehub` helper, and place the community skin-type and skin-issue datasets in the paths listed in `model_service/training/README.md`.
4. Run `python -m model_service.training.train -model all`.
5. Run `python -m pytest tests/ -v` to verify.

Random seeds are fixed to 42 for Python, NumPy, and PyTorch. Results are reproducible up to CPU-specific floating-point non-determinism, which we observed to change the third decimal of the reported accuracies at worst.

Ingredient reference

The recommendation engine draws from a curated vocabulary of skincare ingredients. Table D.1 summarises the most frequently used ones and the situations in which they are recommended. This table is provided as a reader's reference; the canonical source of truth is `backend/blp/rules.json`.

Ingredient	**Typical recommendation**	**Rationale**
Salicylic acid (BHA)	Mild-to-moderate acne, oily skin, enlarged pores	Oil-soluble; penetrates sebum and clears pores.
Benzoyl peroxide	Moderate-to-severe acne, spot treatment	Antibacterial against *C. acnes*; can be irritating.
Niacinamide	Oily skin, sensitivity, redness	Reduces sebum output and inflammation; well tolerated.
Hyaluronic acid	Dry or dehydrated skin	Humectant; binds water in the stratum corneum.
Ceramides	Dry or compromised skin barrier	Rebuild the lipid bilayer of the stratum corneum.
Retinol / retinoids	Wrinkles, uneven texture, non-inflammatory acne	Accelerate cell turnover; contraindicated in pregnancy.
Azelaic acid	Dark spots, sensitive-skin acne	Anti-inflammatory and mild pigment-inhibiting.
Vitamin C (L-ascorbic)	Dark spots, uneven tone	Antioxidant; brightens hyperpigmentation.
Alpha-hydroxy acids (AHA)	Dull skin, mild texture concerns	Exfoliates surface layer; increases photosensitivity.
SPF sunscreen	Every rule that recommends actives	Non-negotiable for anyone using retinoids, AHA, or BHA.
Heavy occlusives (avoid)	Oily skin, active acne	Trap sebum and worsen breakouts.
Fragrance (avoid)	Sensitive skin, active acne	Common irritant; rarely necessary.
Coconut oil (avoid)	Acne-prone or oily skin	Highly comedogenic.

Table D.1: Frequently used ingredients in the CARA recommendation vocabulary.

Chapter E

Glossary

BLP Business Logic Programming — our internal name for the rule engine that converts predictions into recommendations.

CLAHE Contrast-Limited Adaptive Histogram Equalisation — a local contrast normalisation technique used in preprocessing.

EfficientNet A family of convolutional backbones that jointly scale depth, width, and input resolution. We use the smallest member, EfficientNet-B0.

Grad-CAM Gradient-weighted Class Activation Mapping — a technique for visualising which pixels contributed to a prediction. Referenced in Future Work.

MediaPipe A Google library for real-time perception tasks including face detection. Considered as an alternative to Haar cascades.

MobileNet A family of lightweight CNN backbones designed for mobile inference. Compared against EfficientNet in Table 8.3.

ONNX Open Neural Network Exchange — a portable interchange format for trained models. Referenced in Future Work as the path to mobile inference.

Softmax The final normalisation function of a classification head; converts logits to a probability distribution.

Transfer learning Initialising a network with weights pretrained on a large dataset (ImageNet in our case) and fine-tuning it on the target task.

Two-phase training Our recipe: train the head with the backbone frozen, then unfreeze the backbone with a lower learning rate.

Uncertain (report field) The state in which the frontend replaces a predicted label with a neutral message because the model's confidence was below the per-model floor.